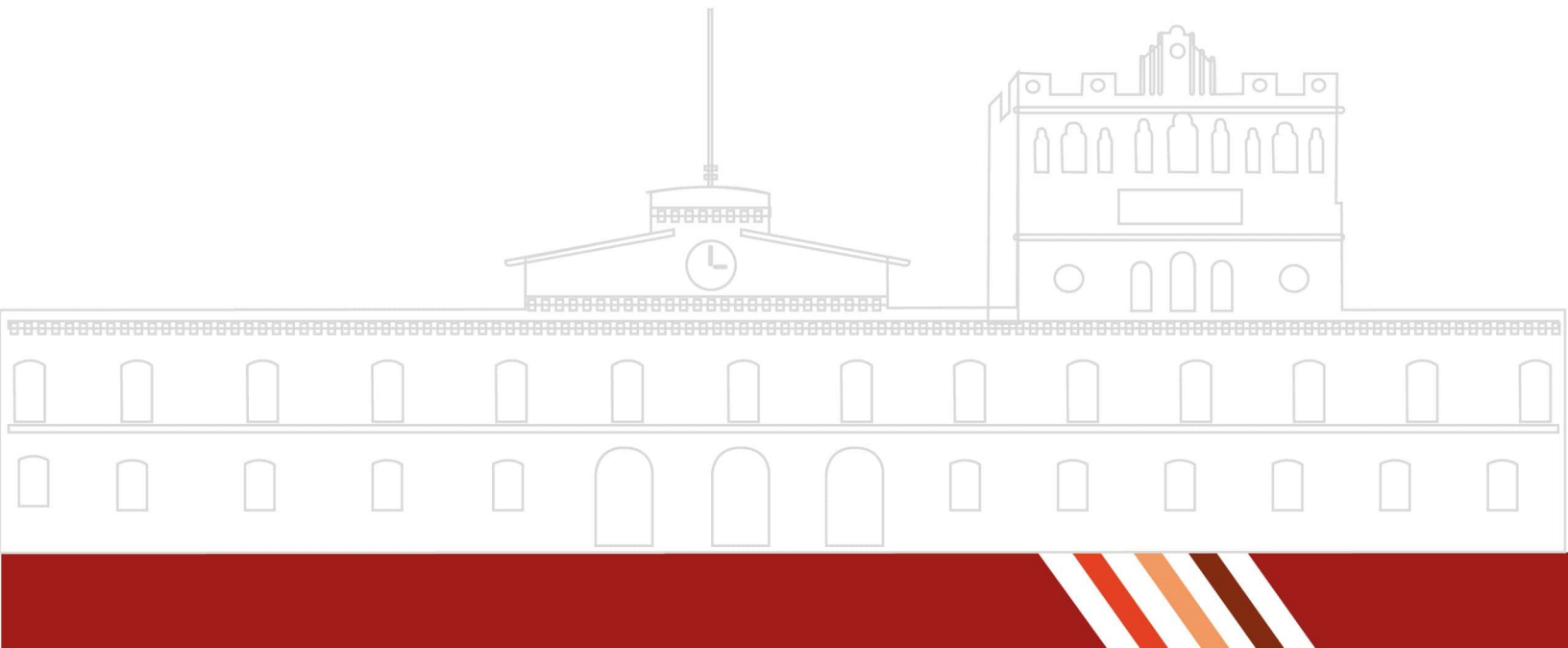


REPORTE DE PRÁCTICA NO. 1

Analisis de expresiones regulares

ALUMNO: Javier Alejandro Pineda Flores
Dr. Eduardo Cornejo-Velázquez



1. INTRODUCCION

El analisis lexico es una de las etapas fundamentales dentro del procesamiento de lenguajes formales, ya que consiste en identificar y clasificar los componentes minimos con significado (tokens) que conforman una cadena de entrada, tales como numeros, palabras, identificadores y simbolos. Esta clasificacion se sustenta en las expreciones regulares, una herramienta matematica que permite describir de manera precisa y compacta los patrones que debe cumplir cada tipo de token, y que constituye la base sobre la que operan los analizadores lexicos, compiladores e interpretes

2. MARCO TEORICO

1. Lenguajes formales

La teoría de lenguajes formales estudia los lenguajes desde un punto de vista matemático y computacional. Un lenguaje formal está formado por un conjunto de cadenas construidas a partir de un alfabeto determinado. Estos conceptos son fundamentales en áreas de la informática como los compiladores, analizadores léxicos y procesamiento de cadenas [?].

Dentro de los lenguajes formales se encuentran los lenguajes regulares, los cuales pueden representarse mediante expresiones regulares y reconocerse utilizando autómatas finitos [?].

2. Expresiones regulares

Una expresión regular es una representación formal utilizada para describir patrones dentro de un conjunto de cadenas. Para construirlas se utilizan símbolos y operadores que permiten combinar diferentes expresiones. Entre los operadores principales se encuentran la unión ($\cup$), la concatenación y la clausura de Kleene (*). También pueden utilizarse paréntesis para agrupar elementos y establecer la prioridad de las operaciones [?].

3. Automatas finitos

Los autómatas finitos son modelos matemáticos utilizados para reconocer lenguajes regulares. Existe una relación directa entre las expresiones regulares y los autómatas finitos, ya que una expresión regular puede representar un lenguaje que posteriormente puede ser reconocido mediante un autómata [?].

Esta relación permite comprender que una expresión regular no solamente representa un patrón de texto, sino que también puede utilizarse como una descripción formal de un lenguaje.

4. Expresiones regulares en Java

Java proporciona herramientas para trabajar con expresiones regulares mediante el paquete `java.util.regex`. Entre sus principales clases se encuentran `Pattern` y `Matcher`.

La clase `Pattern` permite compilar una expresión regular, mientras que `Matcher` permite comprobar y localizar coincidencias entre el patrón y una cadena de texto [?].

Por ejemplo:

```
Pattern patron = Pattern.compile("a*b");
Matcher matcher = patron.matcher("aaab");
boolean resultado = matcher.matches();
```

En este caso, el programa comprueba si la cadena `aaab` cumple con el patrón `a*b`.

5. Analisis de expresiones regulares

El análisis de una expresión regular consiste en identificar los símbolos, operadores y agrupaciones que la componen para determinar su estructura y significado. Un programa dedicado a esta tarea puede recibir una expresión como entrada, analizar sus componentes y determinar si cumple con las reglas establecidas.

Por lo tanto, la práctica desarrollada en Java permite aplicar los conceptos de lenguajes regulares, expresiones regulares y autómatas finitos mediante un programa capaz de procesar y analizar expresiones, relacionando los fundamentos teóricos de la teoría de autómatas con su implementación computacional.

Bibliográfica

@bookCantu2015, author = Thelma Guadalupe Cantú Treviño and María Guadalupe Mendoza García, title = Teoría de Autómatas. Un enfoque práctico, publisher = Pearson Educación, address = México, year = 2015, isbn = 9786073230902

@bookGaudio2022, author = Elena Gaudio Vázquez and Tomás García Sáiz, title = Introducción a la teoría de autómatas, gramáticas y lenguajes, publisher = Editorial Universitaria Ramón Areces, edition = 2, year = 2022

3. HERRAMIENTAS EMPLEADAS

Describir qué herramientas se han utilizado...

1. Java: lenguaje de programación utilizado para desarrollar el programa encargado de analizar las expresiones regulares.
2. Apache NetBeans: entorno de desarrollo integrado (IDE) utilizado para escribir, editar, compilar y ejecutar el programa en Java.

4. DESARROLLO

Análisis de requisitos

El programa debe cumplir con los siguientes requisitos:

1. Permitir al usuario ingresar libremente una cadena de texto en un área de escritura.
2. Clasificar los componentes de la cadena en cinco categorías: números enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos.
3. Ofrecer dos métodos de clasificación sobre la misma cadena: uno mediante estructuras de datos y recorrido manual de caracteres, y otro mediante expresiones regulares.
4. Permitir limpiar el contenido de las áreas de texto para iniciar un nuevo análisis.

Expresiones regulares utilizadas

En la Tabla 1 se presenta el conjunto de expresiones regulares empleadas para clasificar cada tipo de token reconocido por el programa. En la Figura 1 se presenta la interfaz gráfica del programa en su estado inicial.

Table 1: Expresiones regulares utilizadas para la clasificación de tokens.

Categoría	Expresión regular
Número entero	[0-9]+
Palabra minúscula	[a-z]+
Palabra mayúscula	[A-Z]+
Identificador	[a-zA-Z_][a-zA-Z0-9_]*
Símbolo	[^a-zA-Z0-9_\s]

Clasificación mediante estructuras de datos

En la Figura 2 se presenta el resultado obtenido al presionar el botón **ANALIZAR**, el cual recorre carácter por carácter la cadena de entrada y agrupa los tokens en listas según sus propiedades, evaluadas con métodos como `Character.isDigit`, `Character.isLowerCase` y `Character.isUpperCase`.

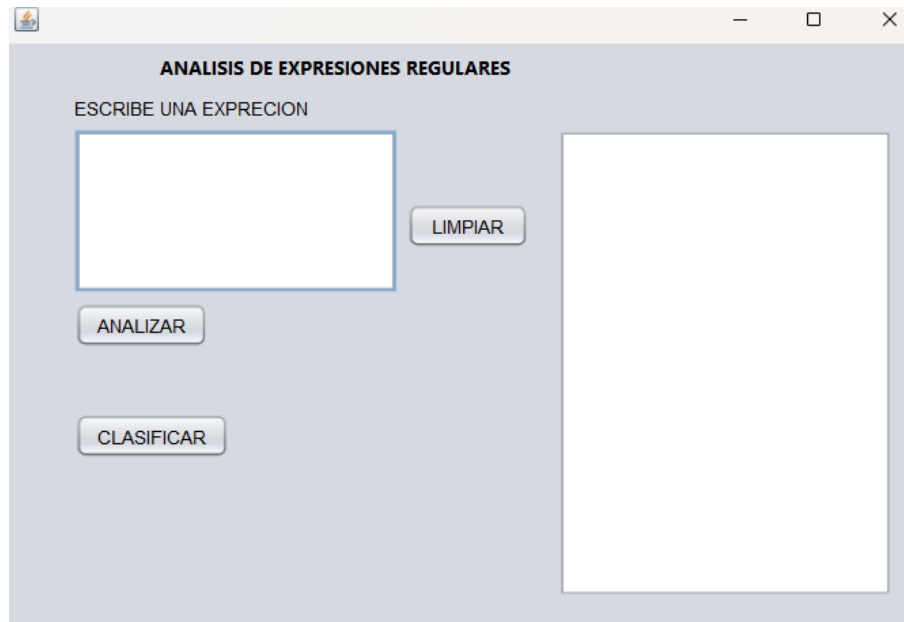


Figure 1: Interfaz gráfica del analizador de expresiones regulares.

Clasificación mediante expresiones regulares

Como alternativa al método anterior, el botón **CLASIFICAR** utiliza las clases `Pattern` y `Matcher` del paquete `java.util.regex` para tokenizar la cadena y determinar la categoría de cada token comparándolo contra las expresiones regulares de la Tabla 1.

En el Listado 1 se presenta el fragmento de código correspondiente a este método.

Listing 1: Clasificación de tokens mediante expresiones regulares.

```
Pattern patronTokens = Pattern.compile(
    "[a-zA-Z_][a-zA-Z0-9_]*|[0-9]+|[^\\"sa-zA-Z0-9_]" );
Matcher m = patronTokens.matcher(cadena);

Pattern pEntero = Pattern.compile("^[0-9]+$");
Pattern pMinuscula = Pattern.compile("^[a-z]+$");
Pattern pMayuscula = Pattern.compile("^[A-Z]+$");
Pattern pIdentificador = Pattern.compile("^[a-zA-Z_][a-zA-Z0-9_]*$");

while (m.find()) {
    String token = m.group();
    String tipo;
    if (pEntero.matcher(token).matches()) {
        tipo = "Numero-entero";
    } else if (pMinuscula.matcher(token).matches()) {
        tipo = "Palabra-minuscula";
    } else if (pMayuscula.matcher(token).matches()) {
        tipo = "Palabra-mayuscula";
    } else if (pIdentificador.matcher(token).matches()) {
        tipo = "Identificador";
    } else {
        tipo = "Simbolo";
    }
}
```

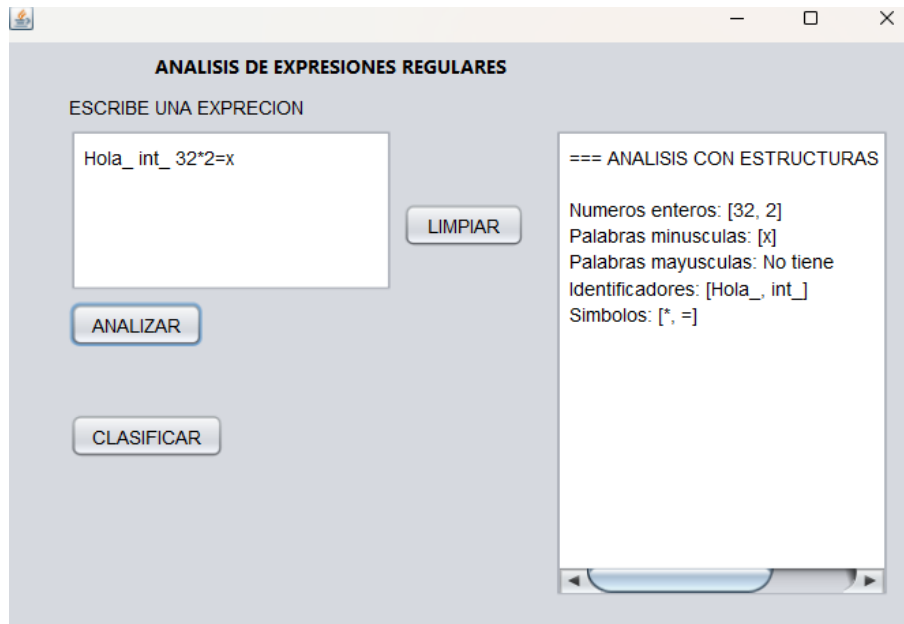


Figure 2: Resultado de la clasificación mediante estructuras de datos.

Finalmente, en la Figura 3 se muestra el resultado obtenido con este método para la misma cadena de prueba, donde se despliega el tipo detectado para cada token de forma individual.

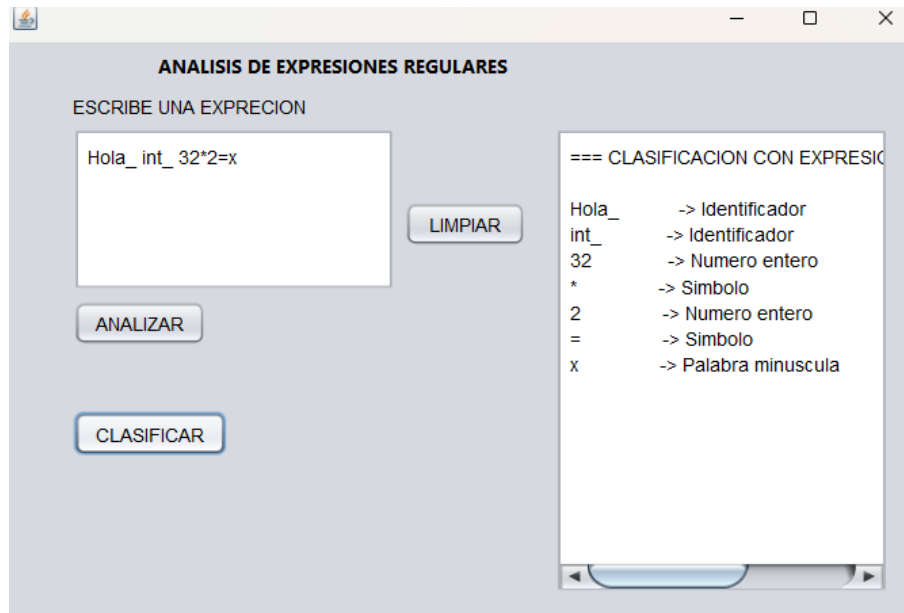


Figure 3: Resultado de la clasificación mediante expresiones regulares.

5. CUESTIONARIO

1. 1. ¿Cuál es la utilidad de las expresiones regulares en la identificación de palabras de un lenguaje?

Las expresiones regulares sirven para identificar y clasificar diferentes elementos dentro de una cadena de texto. En el programa se utilizaron para reconocer números enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos. Por ejemplo, la expresión `[0-9]+` permite identificar una cadena formada únicamente por números, mientras que `[a-z]+` identifica palabras escritas en minúsculas. Esto facilita el análisis de los elementos que forman una cadena.

2. 2. ¿Cuáles elementos del lenguaje de programación utilizaste para la implementación de las estructuras de datos?, ¿Cómo las utilizaste?

Se utilizaron principalmente listas (`ArrayList`), ciclos `while`, estructuras condicionales `if-else` y la clase `StringBuilder`. Las listas se utilizaron para almacenar por separado los números, palabras minúsculas, palabras mayúsculas, identificadores y símbolos encontrados en la cadena. El ciclo `while` permitió recorrer la cadena carácter por carácter, mientras que las condiciones `if-else` determinaron a qué categoría pertenecía cada elemento. Finalmente, `StringBuilder` se utilizó para organizar y mostrar los resultados.

3. 3. ¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos?

Primero se obtiene la cadena que introduce el usuario. Después, el programa la recorre para identificar sus diferentes elementos. Se ignoran los espacios y se agrupan los caracteres que pueden formar una palabra, número o identificador. Posteriormente, se revisa cada elemento para determinar a qué categoría pertenece y se almacena en una lista correspondiente. En la segunda forma de análisis, se utilizan expresiones regulares para encontrar y clasificar los elementos. Finalmente, el programa muestra los resultados y el total de elementos encontrados.

4. 4. ¿Qué elementos del lenguaje de programación te permitieron implementar expresiones regulares?

Para implementar las expresiones regulares se utilizaron principalmente las clases `Pattern` y `Matcher` de Java, pertenecientes a la biblioteca `java.util.regex`. `Pattern` permite definir el patrón que se quiere buscar y `Matcher` permite comparar ese patrón con el texto. También se utilizaron condiciones `if-else` para determinar qué tipo de elemento había sido encontrado.

5. 5. ¿Qué condiciones o recomendaciones se deben seguir al escribir las expresiones regulares?

Al escribir una expresión regular es importante definir claramente qué se quiere identificar, utilizar correctamente los caracteres especiales y tener cuidado con los símbolos que tienen un significado específico. También se debe comprobar que el patrón no sea demasiado amplio, porque podría identificar elementos que no corresponden a la categoría buscada. Por ejemplo, `[0-9]+` permite identificar números de uno o más dígitos, mientras que `[a-z]+` solamente permite letras minúsculas. Por eso, es recomendable probar las expresiones con diferentes ejemplos para comprobar que realmente reconocen lo que se necesita.

6. CONCLUSIONES

Al realizar esta práctica pude darme cuenta de que volver a programar después de un tiempo sin hacerlo no fue tan sencillo como esperaba, ya que al principio tuve que recordar varias cosas que ya había aprendido anteriormente.

Uno de los aprendizajes más importantes fue entender mejor cómo se pueden analizar y clasificar diferentes tipos de datos dentro de una cadena. También reforcé el uso de listas, ciclos, condiciones y expresiones regulares.

La práctica permitió cumplir con la finalidad de desarrollar un programa capaz de analizar una cadena de texto y clasificar sus elementos mediante estructuras de datos y expresiones regulares. De esta manera, el programa permite identificar números enteros, palabras, identificadores y símbolos, además de mostrar los resultados de forma organizada.

Referencias Bibliográficas

References

- [1] Oracle. (2024). Tutorial de Java. *Documentación de Java*. Oracle. Disponible en: <https://docs.oracle.com/javase/tutorial/>
- [2] Oracle. (2024). Expresiones regulares. *Tutorial de Java*. Oracle. Disponible en: <https://docs.oracle.com/javase/tutorial/essential/regex/>
- [3] Oracle. (2024). El marco de colecciones de Java. *Tutorial de Java*. Oracle. Disponible en: <https://docs.oracle.com/javase/tutorial/collections/>
- [4] Schildt, H. (2019). *Java: A Beginner's Guide*. McGraw-Hill Education.
- [5] Deitel, P. J.; Deitel, H. M. (2017). *Java: Cómo programar*. Pearson Educación.